

CS 40700 Project Charter

Team:

Aagam Bapna - abapna@purdue.edu

Cheng-En Lee - lee4649@purdue.edu

Anthony McCrovitz - amccrov@purdue.edu

Daniel Benes-Magana - dbenesma@purdue.edu

Jaime Xu - xu1545@purdue.edu

Problem Statement

Scanned documents hold a lot of useful information, but tables and page breaks make it hard to pull that information out or search it. This project lets users upload documents in bulk, turns them into searchable structured data, and shows exactly where on the page each result came from.

Many document-parsing platforms require users to submit each document manually one at a time, which can be time consuming and potentially cause rate limitations. This project would allow users to systematically parse large volumes of documents, a capability that is not otherwise provided by similar systems.

Project Objectives

- Build a web tool where users can systematically parse large batches of scanned documents, including multi-page PDFs and images scans.
- Provide an ingestion pipeline that performs OCR and extract document contents into text data
- Segment document layouts into semantic regions (headers, paragraphs, figures, tables) and reconstruct split tables spanning across page breaks using spatial heuristics and layout parsing models.
- Render an interactive document viewer overlaying bounding boxes on the original document, allowing users to inspect and verify extracted JSON side-by-side with the source.
- Manage background OCR and inference tasks with load balancing, distributed job queuing, progress tracking, and fault tolerance.
- Provide search capabilities where user can locate text data across previously scanned documents

Stakeholders

Users: Professionals who need to extract, audit, and analyze tabular and visual data in large volumes of scanned reports (such as financial 10-K filings, scientific publications, and legal filings) without manual data entry.

Software Developers: Cheng-En Lee, Aagam Bapna, Jaime Xu, Daniel Benes-Magana, Anthony Michael McCrovitz

Project Owners: Cheng-En Lee, Aagam Bapna, Jaime Xu, Daniel Benes-Magana, Anthony Michael McCrovitz

Deliverables

- A secure website to upload multiple documents and track their processing status.
- Reads text from scanned pages and saves the exact position of every word.
- Breaks pages into headers, paragraphs, figures, and tables, creating a detailed JSON file.
- Shows the original page next to the extracted data. Allows users to click and highlight specific regions to check for errors.
- A task queue where background workers process pages, retry failures, and provide live progress updates.
- A full-text search across processed documents that links back to the exact page and region, with built-in filters.
- A complete test suite covering unit tests, layout parsing integration tests, and continuous deployment workflows.

Technology Stack:

- Frontend:
 - [Next.js](#)
 - TypeScript
 - Tailwind CSS
 - [PDF.js](#) for custom bounding-box rendering
- Backend:
 - FastAPI for handling API requests and managing jobs
 - valkey for asynchronous job queuing
 - PostgreSQL for user data storage
 - AWS S3 for document storage
- Document Processing:

- Python-based node that parse text data from documents and reach out to inference services
- Inference:
 - `vllm` or `sglang` for serving the OCR/VLM and other machine-learning models
- Search:
 - `Elasticsearch`
 - PostgreSQL full-text search
- Hosting and Deployment:
 - Vercel (Frontend)
 - Docker
 - Cloud compute instances for backend workers
- Task Queue and Distributed Execution
 - Celery
 - Redis

Commented [1]: some options for inference engine. may also be worth looking at llama.cpp, but these are "production grade"

Commented [2]: I have worked with deploying llms on vllm before. I got a dgx spark so we might be able to test run OCR locally. Might need to figure out a way to deploy it for free.
1 total reaction
Daniel B reacted with 👍 at 2026-08-29 12:50 AM UTC

Commented [3]: this may be a little too heavy for what we want to do, but it's the first thing that came to mind

CS 30700 Projects

Aagam Bapna: A website that helps students find places to study and people to study with. Students can see which rooms on campus are open or crowded based on reports from other students, set up study sessions for a class, and share notes with everyone taking that class. An assistant answers questions using those shared notes, so answers match what was taught in the course. Students can also message each other and add friends, and clubs can post events. The goal is to make learning easier, not alone.

<https://github.com/AagamBapna/BoilerSpace>

Daniel Benes-Magana: Centralized Lost-and-Found system for campus. Beyond a traditional lost-and-found “catalog” with best-effort keyword generation, search, and location tracking. It has a pre-registration system; users get a qr code, and if the item is lost, they get notified when it’s found and returned to a desk.

Jaime Xu: Mobile app designed to assist Purdue students and staff in choosing the best dining hall for their meals by providing real-time updates on crowd levels, wait times, and menu options. The app caters to users with various dietary needs, offering features like tracking dietary restrictions, current occupancy levels, and proximity to dining courts, all of which play a crucial role in meal planning decisions.

Cheng-En Lee: Mobile dating app targeted at Purdue students, designed for healthier, long-lasting relationships. The app onboards Purdue students, creates matches, and hosts events tailored for Purdue students. To promote users to appreciate their partner, this app features a matchmaking cycle where users are provided with a match only on a weekly basis, based on the profile preferences provided in earlier app use. This design ensures that each user focuses on building bonds with the current partner instead of with multiple partners at a time.

<https://github.com/BoilerBond>

Anthony McCrovitz: A mobile app designed to be a real-time parking solution for Purdue University. The app onboards users then provides live availability updates of the parking and shows a calendar for events at the University affecting available parking and providing ML-based occupancy predictions for the parking lots at Purdue. The app also has calendar integration, and intelligent routing to help students and visitors find parking spots efficiently.

<https://github.com/Neel-Vachhani/Purdue-Parking-App>